

Engineering drawings - Group 22

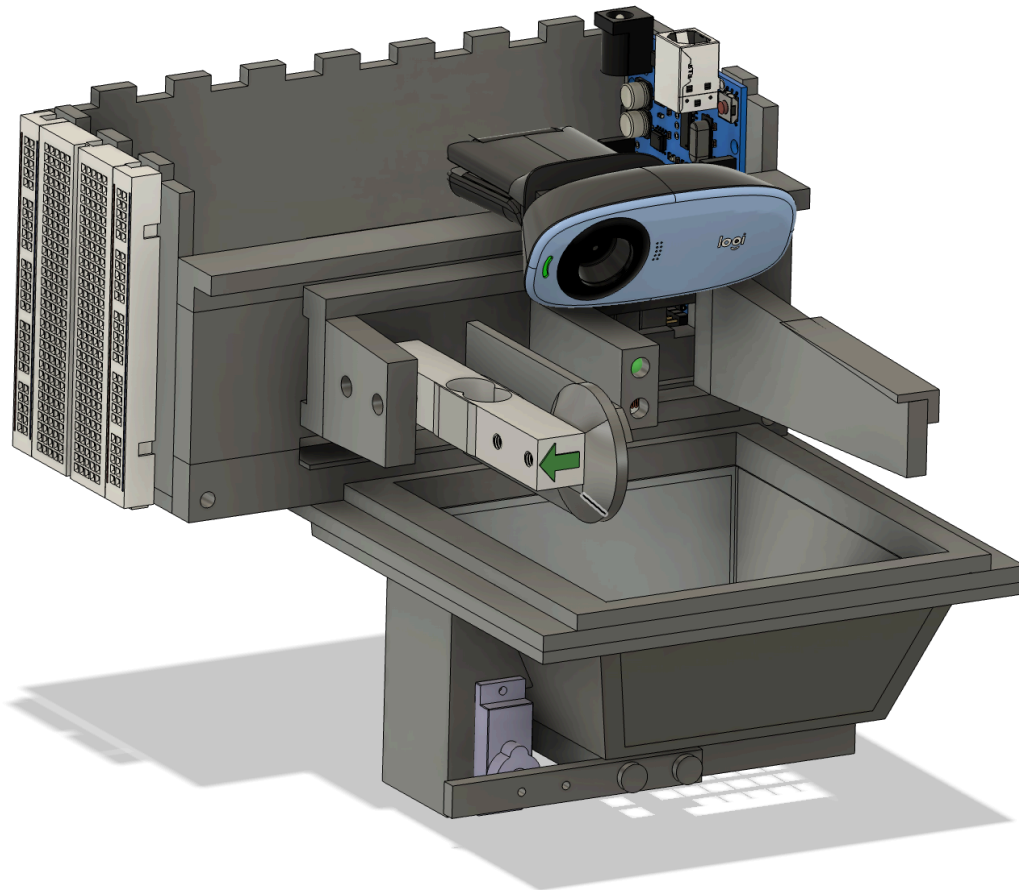


Figure 1 - Gripper Main Assembly

De Foucauld Solvène Sciper : 358483

Naili Sarah

Sciper : 363174

Villa César

Sciper : 334551

Ricard Luc

Sciper : 362858

De Francesco Giandomenico Sciper : 345299

Table of Contents :

Table of Contents :	1
Introduction :	2
Two important parts of our project :	3
First part : Gripper center support	3
Second part : Gripper left rack	4
Bill of materials :	5
System block diagram :	6
Electronics schematics :	7
Servo motor :	7
Mini servo motor :	8
Control and Decision-Making Representation :	9
Pseudo-code for the Python Code :	9
Pseudo-code for the Arduino Code :	10
Gantt Chart :	11
Appendix, 2D CAD :	12

Introduction :

During this semester, we developed a gripper system designed to automate the process of picking raspberries using an Arduino. The gripper operates through a servo motor that drives a gear mechanism, which in turn moves two racks. This configuration enables the opening and closing of the gripper's arms. To identify ripe raspberries, we use a webcam combined with a Python code. We first used a home-made color sensor but we remarked that the camera is more reliable. The color sensor is still on the design but it wasn't used during the test. At the press of a key on the computer, the harvesting process starts with the first step: the detection of ripeness. Once a raspberry is detected as ripe, the system activates a servo motor to grip it thanks to our Arduino. Of course, if the raspberry is detected as unripe, nothing happens.

When the gripper reaches the precise angle we determined based on numerous previous tests, the motor stops for a certain amount of time allowing us to lower the robotic arm to collect the fruit. As you can see on our 3D render, we first thought about using a load cell to know when to stop the arms. But, at the end we remarked that it was not useful and could cause more problems to our system. We kept it for the design but it wasn't operational.

The gripper then releases the raspberry into a collection container attached to the system by reopening the arms. This process is repeated for the subsequent raspberries, ensuring a continuous harvesting cycle. At the end of the operation, a trapdoor mechanism is triggered by pressing a key on the computer, releasing all the collected raspberries into a final container thanks to a micro servo motor.

This project has been really interesting because it combines mechanical and electronic systems to create an efficient and automated solution for raspberry harvesting, demonstrating the potential of integrated control systems in agricultural applications.

In this document, we will present two CAD diagrams, a bill of materials, a system block diagram, the electronics schematics, a pseudo-code of our Arduino C++ code and of our python code for the camera and our final Gantt Chart.

Two important parts of our project :

During this project, we designed a lot of parts and prototypes for our parts. In this section, we will present two parts that we find useful and interesting.

First part : Gripper center support

The pinion rack mechanism is supported by a centerpiece, divided into three sub-parts: bottom, center, and top. The center piece ensures the secure positioning of the top and bottom racks while also housing our light sensor (its usage is detailed in the final report). It is constructed from a primary PETG plate, which includes space for the light sensor cover and electronics storage. The top and bottom rack slots are designed with a 0.2 mm tolerance to allow smooth gliding without interference, although some sanding was required to address minor 3D printing imperfections.

The 2D CAD of this part can be found in the annex at the end of this document.

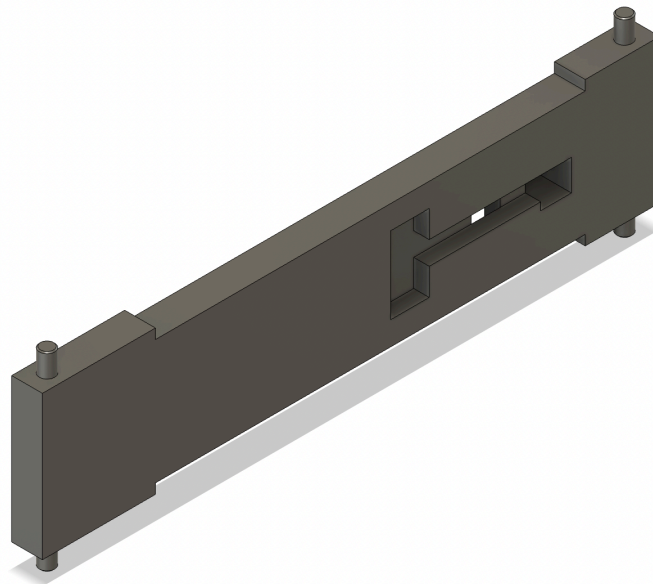


Figure 2 - Gripper center support

Second part : Gripper left rack

The left rack arm is a central part of our prototype, forming one of the two arms needed to grip the raspberries. This component uses a rack and pinion to convert the rotary motion of the servo motor into a linear motion, bringing the gripper arms together and exerting gentle pressure on the fruit to loosen it gently. The design of the rack incorporates regular, standardised and uniform teeth on its active surface. These features ensure smooth interaction with the pinion, guaranteeing jolt-free movement. This mechanism offers great precision in movement and effectively manages the pressure exerted on the fruit, avoiding damage to the fruit. In addition, this part has been made from PETG, a material with low friction, which is essential for ensuring that the rack slides smoothly in its rails. The precision of its manufacture, with a tolerance of just 0.2 mm, also contributes to its smooth operation. Lightweight (45 g) but rigid, it offers excellent mechanical strength, adapted to the stresses to which it is subjected, while remaining economical to produce on a large scale thanks to its streamlined design.

The 2D CAD of this part can be found in the annex at the end of this document.

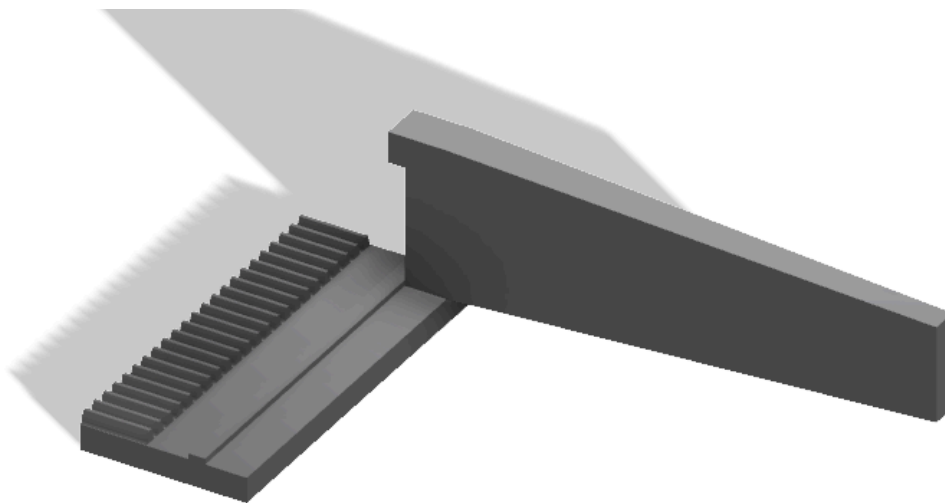


Figure 3 - Gripper_left_rack

Bill of materials :

During our project, we designed our custom-made parts with *Fusion 360*. Then, we 3D-printed them in **PETG**. For the other electronic components, we used commercial parts distributed by the TAs.

Here, we have a list of our parts with their origin, their material and their function.

Part Name	Source	Material	Function
Gripper Arms	Custom-made	PETG	Opens and closes to grip and release raspberries.
Gear	Custom-made	PETG	Transfers rotational motion from the servo motor to the racks.
Racks	Custom-made	PETG	Translates rotational motion into linear motion for gripper arm movement.
Trapdoor Mechanism	Custom-made	PETG	Releases all collected raspberries into the final container.
Collection Container	Custom-made	PETG	Temporarily holds raspberries after they are picked.
Frame	Custom-made	Wood and PETG	Provides structural support for the gripper and its components.
Arduino Board	Commercial	Electronic Components	Controls the actuators, and overall operation of the gripper system.
Servo Motors	Commercial	Electronic Components	Provides rotational motion to operate the gear, the racks and the trapdoor.
Webcam C270 HD	Commercial	Electronic Components	Detects the ripeness of the raspberry.
Power supply	Commercial	Electrical Components	Provides power to the system (6V).
Screws and nuts	Commercial	Metal	Securely assemble and fasten all mechanical components together.

Figure 4 - Bill of materials

System block diagram :

Here we have the block diagram of our system.

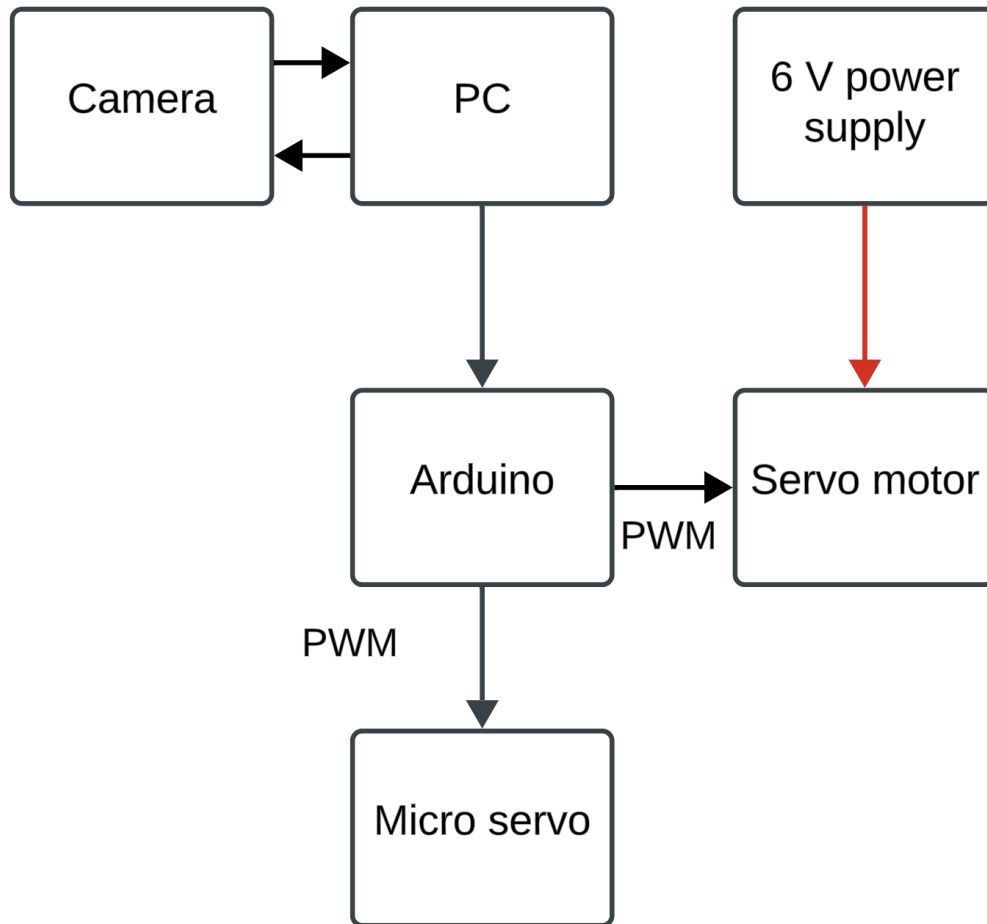


Figure 5 - System block diagram

Electronics schematics :

Servo motor :

Here is the electronic schematic of our Servo motor used to pick the raspberries. We use a servo motor combined to an external power source of six volts to ensure a good functioning of our gripper.

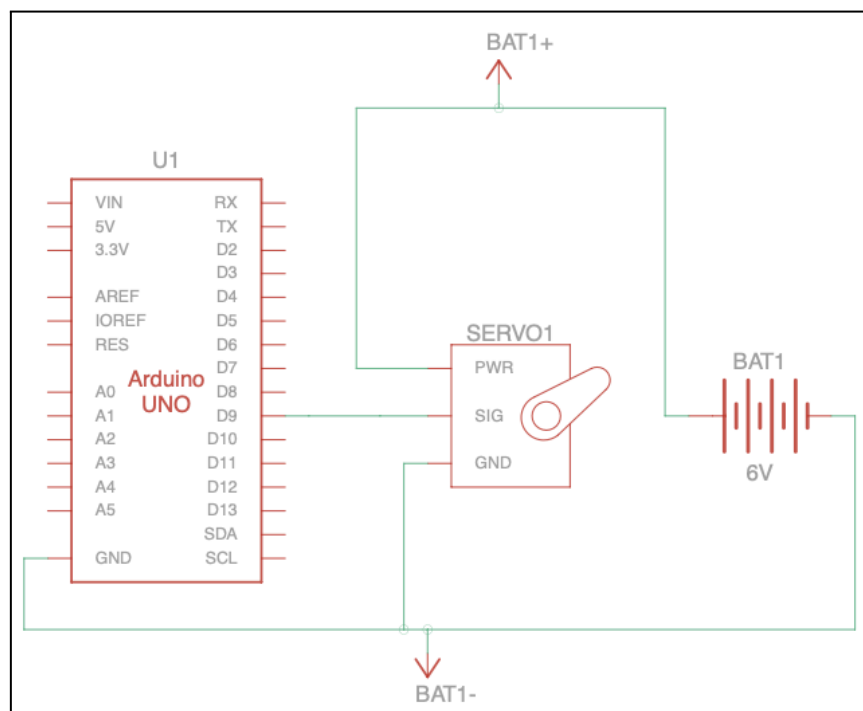


Figure 6 - servo motor electronic schematic

Mini servo motor :

Here is the electronic schematic of our Servo motor used to release the raspberries from our basket to the final location.

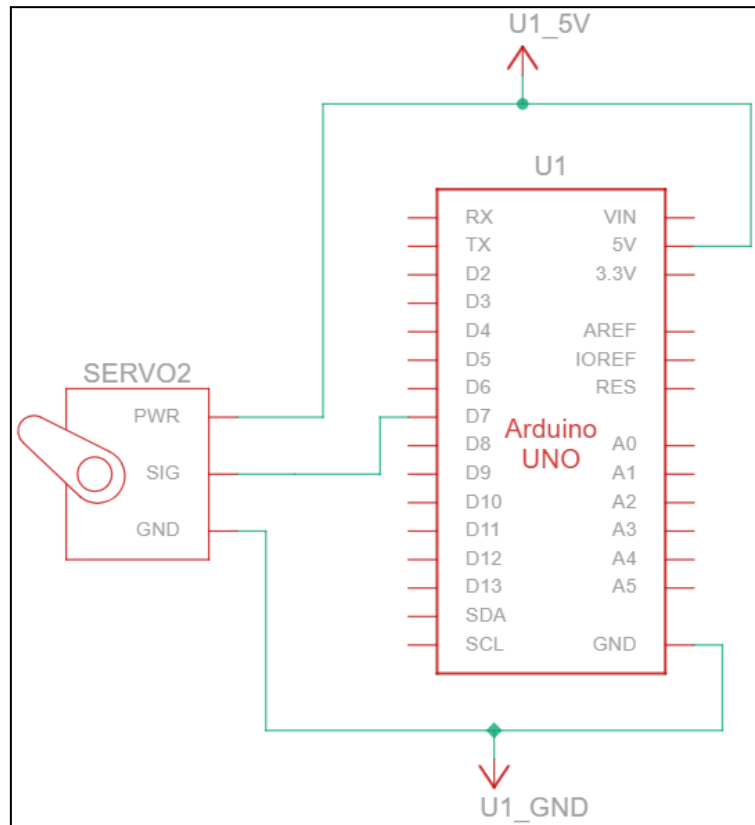


Figure 7 - micro servo motor electronic schematic

Control and Decision-Making Representation :

In this section, we will present the pseudo codes of our system. The first one is the python code and the second one is the Arduino code.

Pseudo-code for the **Python Code** :

START

1. INITIALIZE SYSTEM

- Import required libraries (OpenCV, NumPy, serial, and time).
- Configure the serial connection with the Arduino.
- Initialize the webcam for image capture.

2. DETECT RIPENESS

- Capture an image using the webcam.
- Analyze the central region of the image:
 - Convert the image to the HSV color space.
 - Apply a mask to detect red pixels.
 - Count the red pixels in the masked region.
- IF the red area exceeds the defined threshold:
 - Send the signal 'r' to the Arduino.
 - PROCEED to Step 3.
- ELSE:
 - PROMPT the user to analyze another raspberry and RETURN to Step 2.

3. OPEN HATCH (OPTIONAL)

- IF the user sends the command 't':
 - Send the signal 't' to the Arduino.
 - Wait for feedback from the Arduino to confirm the action.

4. FINALIZE OPERATION

- IF the user sends the command 'q':
 - Exit the program.
- Release the webcam and close the serial connection.

END

Pseudo-code for the Arduino Code :

START

1. INITIALIZE SYSTEM

- Begin serial communication.
- Attach the main servo and secondary servo to the corresponding pins.
- Set the initial position of both servos.
- Print "System Ready."

2. WAIT FOR SIGNAL

- Monitor the serial input for commands from the Python program.
 - IF 'r' is received:
 - PROCEED to Step 3.
 - IF 't' is received:
 - PROCEED to Step 4.

3. GRIP RASPBERRY

- Move the main servo to grip the raspberry.
- Wait for a specified duration.
- Return the servo to its initial position.
- Print "Main servo reset."

4. OPEN HATCH

- Move the secondary servo to open the hatch.
- Wait for a specified duration.
- Return the servo to its initial position.
- Print "Hatch reset."

5. FINALIZE SYSTEM

- Repeat Steps 2–4 as long as commands are received.

END

Gantt Chart :

Here is our final Gantt chart. We have the tasks on the left and the number of the week on the top .

It changes a lot compared to the one we made at the beginning of the project. In fact, we couldn't plan when our prototypes would be ready. Furthermore, at this time we weren't sure of how quickly we would be able to advance. For these reasons, the main differences are the time for the CAD and the Arduino coding. In red, we can see when we decided to stop the detection with the color sensor and when we stopped to work on the load cell.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Arduino learning			■	■	■	■								
Fusion 360 learning	■	■	■											
Concept selection	■	■	■											
First prototype			■	■	■	■	■	■						
Second prototype									■	■	■	■		
Final product													■	■
Tests color sensor							■	■	■	■	■	■	■	
Tests load Cell							■	■	■	■	■	■	■	
Tests camera													■	■
Assembly + testing								■	■	■	■	■	■	■

Figure 8 - Gantt Chart

Appendix, 2D CAD :

You will find the 2D CAD of our two parts on the next two pages.

